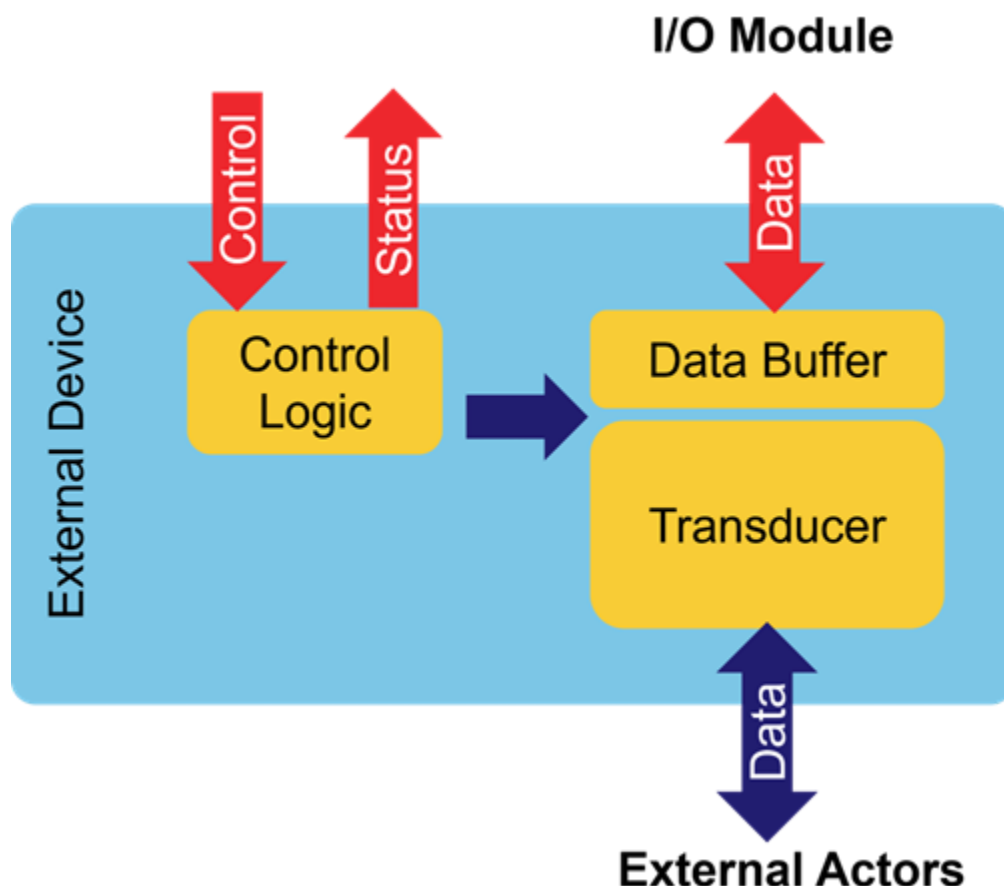


Basics of Input/Output Operations

External Devices:

Input/Output operations between a computer system and its external actors are accomplished using a wide variety of external devices. An external device connects to the computer using an I/O module. A keyboard can be used as an input device to provide information from a computer user to the computer system. Similarly, a monitor is the most often used output device to display information from a computer system to the computer user.

Any external device exchanges control, status, and data information with an I/O module. Control signals from an I/O module to the external device define the function that the external device shall perform. This function could be to write or read data to or from an I/O module, to report device status, or to perform some device-specific internal functions like the positioning of the disk head. The control logic within the external devices receives these control signals and controls the device's functions. Status signals from the external device to the I/O module provide the external device's status to the computer. This state information could be in, for example, device-ready or device-not-ready or other such states.



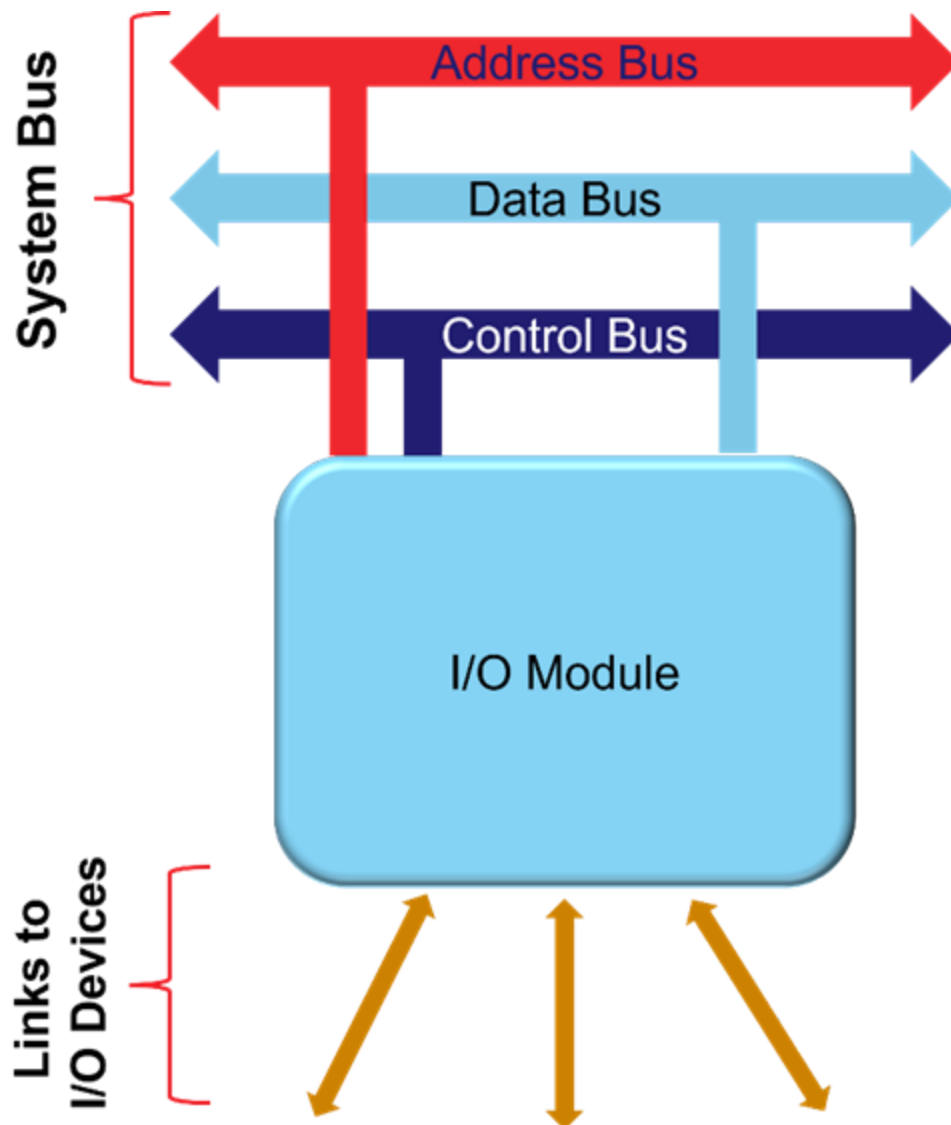
An external device can be classified as one of three types: human-readable, machine-readable, and communication. As the name suggests, human-readable device types are those that are suitable for communicating with the computer user. Examples of Human Readable devices are display monitors or printers. Machine-readable devices are those that are suitable for communicating with equipment.

These device types include storage devices like disks and tapes or monitoring and control devices like sensors and actuators. Lastly, the communication device types are those that are suitable for communication with remote devices. Such devices include modems and communication dongles that can connect to remote devices using different communication technologies.

I/O Modules:

Why do we need an I/O module to connect I/O devices to computers? And why can the I/O devices not be directly connected to the system bus of the computer? The answer to both questions is that there are a variety of I/O devices, each of them having different characteristics. For one, every I/O device has its own method of operation. For a processor to include the logic required to operate and control each I/O device in a different manner would make the logic extremely complex. Also, the I/O devices do not operate at the same speed as the CPU or the memory. Most I/O devices operate at much slower speeds than the CPU, memory, and system bus. In contrast, there are some I/O devices that may even operate much faster than the CPU or the memory. If the I/O devices are directly connected to the system bus, the difference in speeds will make the management of the I/O operations extremely difficult. Lastly, different I/O devices also use different data formats and often also different word lengths than what the computer uses. This requires a format conversion before data can be provided to the CPU or the memory. Due to all of these reasons, we require an intermediary between the computer and the I/O devices – the I/O module.

The diagram here shows a generic model for an I/O module. We can see why the I/O module is an intermediary. On one end, the I/O module connects with the computer's system bus, while on the other end, it links with one or more I/O devices using special-purpose data links.



An I/O module is responsible for maintaining the control and timing required for the processor to communicate with the I/O devices. There is a need to control the transfer of data between the external I/O devices and the internal resources of the computer, like the system bus or the memory. The I/O module fulfills this responsibility.

Communication with the processor or the CPU is the next function of the I/O module. The I/O module must be able to accept and decode commands from the CPU, like READ-DATA or WRITE-DATA, or other commands. It must be able to send or receive data to and from the CPU using the data bus. The I/O module also shares with the CPU the status of the external device, like DEVICE-BUSY or DEVICE-READY.

On the other end, the I/O module must communicate with the I/O device. This includes exchanging data with the device, sending control information to the device, or receiving status information from the device.

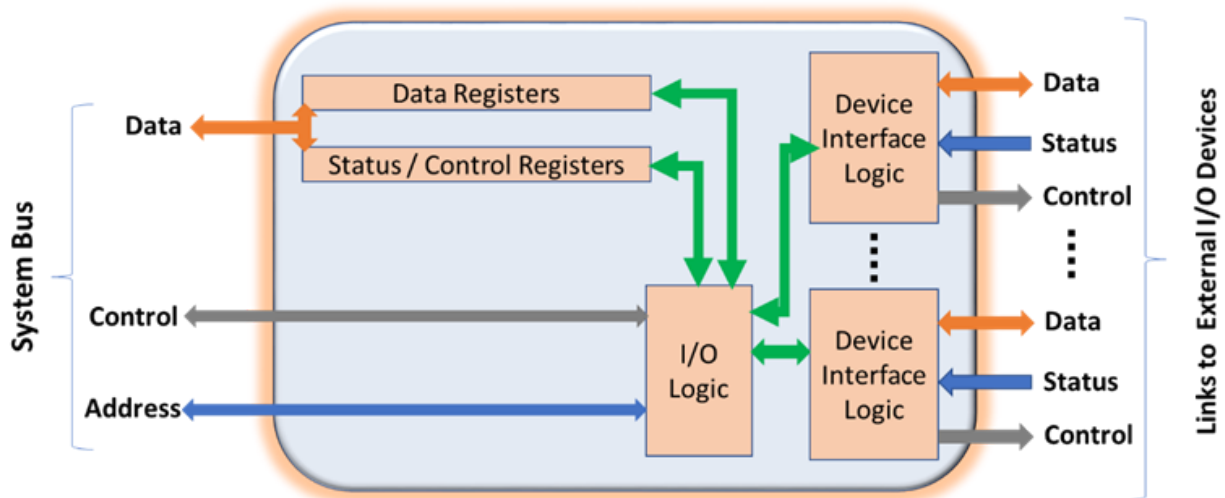
An I/O module also needs to have in-built buffers to perform data buffering. Since the internal computer resources, like the memory and system bus, operate at speeds that are different than the speed of the I/O device; data may need to be buffered within the I/O module temporarily. The I/O

module communicates with the computer at the speed of the system bus, while with the I/O devices, it communicates at a speed that the device supports.

Quite often, the I/O module may also be implementing error detection and reporting to the CPU.

These errors could be mechanical or electrical failures or could even be transmission errors.

Different I/O modules can be quite different from each other in terms of their design and internal structure. Even the number of external I/O devices that a single I/O module handles can differ from one I/O module to another. So, what is depicted in the below diagram is only a simplistic and generic block diagram for an I/O module.



In general, these are the components that you can expect within an I/O module. The most important component here is the I/O logic. It is the I/O logic that receives and handles the commands coming from the CPU. The I/O logic interfaces with the **device interface logic**, which is used to communicate and control the external I/O devices. Data and status buffering is achieved via a set of registers that are included inside an I/O module.

An I/O module can be quite much more complex than depicted here, and it all depends on the functions that an I/O module performs and how much work it offloads from the processor.

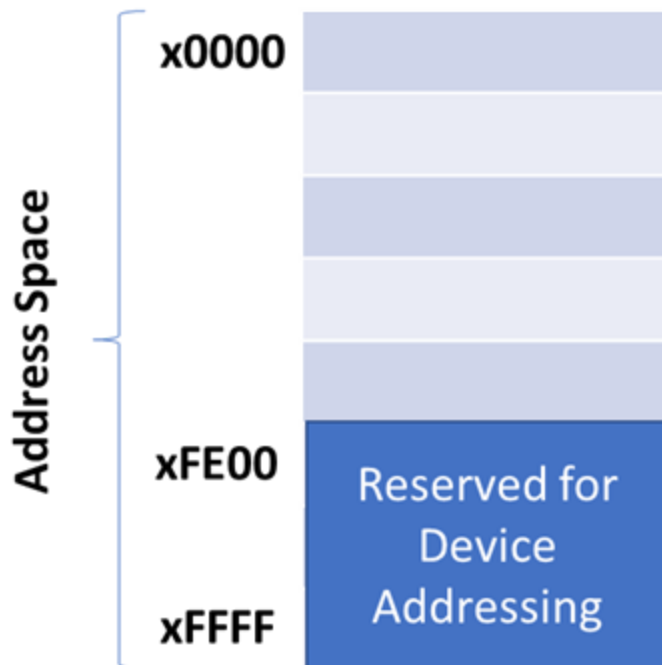
Overview of I/O Concepts:

In terms of I/O concepts, three key concepts are discussed throughout this entire module. These are device addressing, the timing of data transfer, and the control of data transfer.

Device Addressing covers how the external devices are addressed. Like memory read/write operations, dealing with the device is actually equivalent to dealing with its associated registers. So, essentially, device addressing includes how the device registers themselves are addressed or identified. Timing of Data Transfer covers how data transfer timing is managed, considering that the processor and the external device operate at different data transfer rates. Lastly, control of data transfer describes which entity or component controls the entire data transfer—whether the processor or the device itself.

Device Addressing:

Two methods are commonly used to identify device registers. The first method is called memory-mapped I/O, whereas the second method makes use of Special I/O instructions. Memory-mapped I/O is a method that is driven by two key concepts: the use of memory addresses to uniquely identify each device register and the use of data movement type instructions to perform I/O operations. The diagram below describes how memory addresses can be used to uniquely identify each device register.



Every storage location needs an address to be uniquely identified. In memory-read / memory-write operations, we address a memory location using its unique memory address. The range of possible addresses is called the address space. In our diagram, the address space ranges from address x0000 to xFFFF. Not all of these addresses may be required to address memory locations, though. In memory-mapped I/O, a part of this address space is reserved for use in identifying device registers. In other words, every device register is given a unique address from this reserved range of the address space. In fact, it is for this reason that the I/O method gets its name as the memory-mapped I/O method.

Now that a memory address is provided to each device register, normal data movement type instructions can be used to perform I/O operations. It is just like performing read/write operations in memory. With the source address being that of an input device register, the load instruction is equivalent to performing an input operation. Similarly, using a store instruction with the destination address of an output device register results in an output operation. With memory-mapped I/O, no special I/O instructions are needed. I/O operations are performed just like read/write operations in memory.

The other method used for device identification is by using specialized I/O instructions. Unlike memory-mapped I/O, where the data movement type instructions are reused for I/O operations, in this method, dedicated Instructions are provided as part of the processor's ISA and are used for I/O operations. The ISA contains one or more designated Opcodes to be used for I/O operations. The device identifier, as well as the operation, are encoded inside the instruction itself.

Timing of Data Transfer:

Most of the I/O operations are performed at speeds that are quite much slower than the speed at which the processor operates. Hence, managing the timing of data transfer becomes a challenge. Two common methods are used to manage the timing of data transfer: synchronous and asynchronous. In Synchronous mode, the data transfer happens at fixed times, both from the device end as well as the processor end. Thus, data transfer is said to be *time-synchronized*. On the other hand, in the asynchronous mode of data transfer, there is no predictability on when the data will be

available for transfer, and therefore, the device and the CPU must communicate or talk to each other for each data transfer to make sure that no data gets missed. This topic is further discussed later.

Control of Data Transfer:

Three different I/O techniques are commonly used for control of data transfer, and these are called programmed I/O, Interrupt-driven I/O, and direct memory access or DMA.

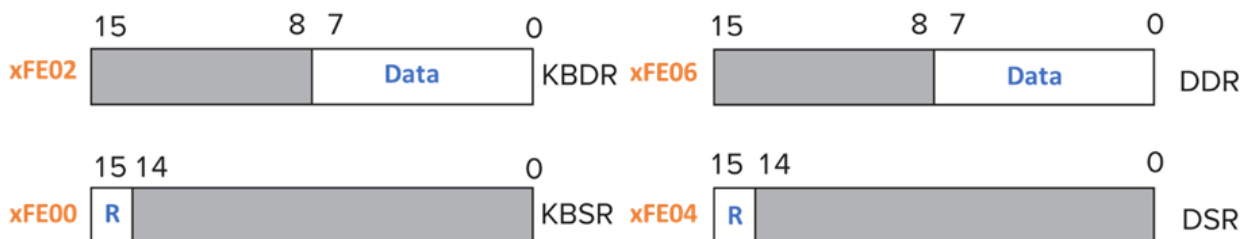
In Programmed I/O and Interrupt-driven I/O, the CPU or the processor is involved in the data transfer from the I/O device to the memory and vice-versa. So, CPU cycles are consumed in performing the I/O operations. On the other hand, as the name suggests, in DMA transfer, data is transferred from the I/O device to memory storage directly, bypassing the CPU. In other words, no CPU cycles are consumed in DMA-based data transfers.

Another differentiating aspect of these I/O techniques is who amongst CPU or Device controls the data transfer. In Programmed I/O, the CPU decides when it wants to perform the I/O operation. The CPU implements a polling-based approach to continuously read the device status register in order to check when the device has input data or is ready to accept output data. On the other hand, Interrupt-driven I/O and DMA are interrupt-based I/O techniques. In this technique, the device sends an interrupt signal to the CPU whenever there is a change in its state, that is, whenever it has some new data to send or it is ready to receive output data. Upon receiving this interrupt, the CPU interrupts the software program it is currently executing to handle the I/O operation before going back to executing the earlier software program. All these three techniques are discussed in more detail later.

Input/Output Devices in LC-3:

In the LC-3, the keyboard is used as an input device, while the display monitor is used as the output device. The keyboard works using two device registers, the keyboard data register, or the KBDR, and the keyboard status register, or the KBSR. While both registers are 16 bits each, only some of the bits of either register are actually used. Bits 0 to 7 of the KBDR are used for storing the Data, while Bit 15 of the KBSR is used for the ready bit. The other bits are unused. Since LC-3 uses memory-mapped I/O, all device registers are given an address from the memory address space. Accordingly, the KBDR is allocated an address of xFE02, while the KBSR is allocated the address xFE00.

Like the keyboard device registers, the display monitor also works using two device registers, the display data register, or the DDR, and the display status register, or the DSR. Again, only some of the 16 bits of either register are used. Bits 0 to 7 of the DDR are used for storing the Data, while Bit 15 of the KBSR is used for the Ready bit. The other bits are unused. The DDR is allocated an address of xFE06, while the DSR is allocated the address xFE04.

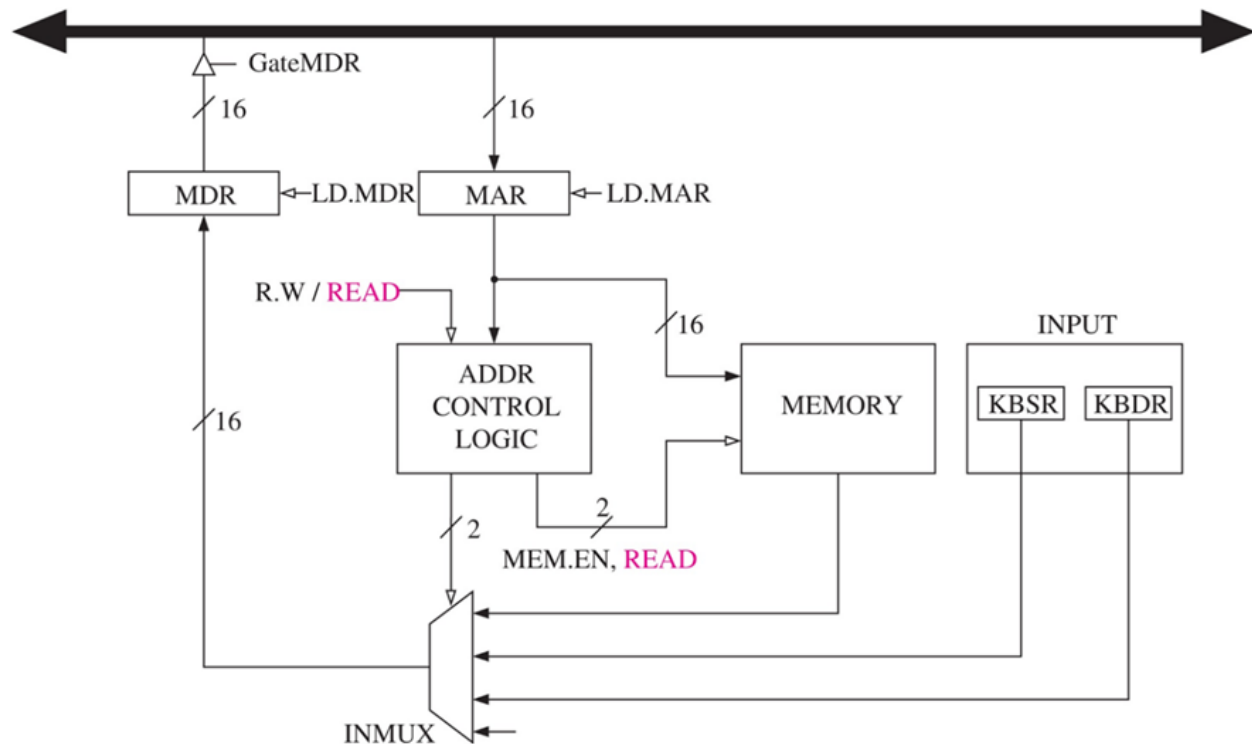


Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond)*, McGraw Hill, 3rd Edition

Let us discuss how the keyboard input device works using the memory-mapped I/O approach.

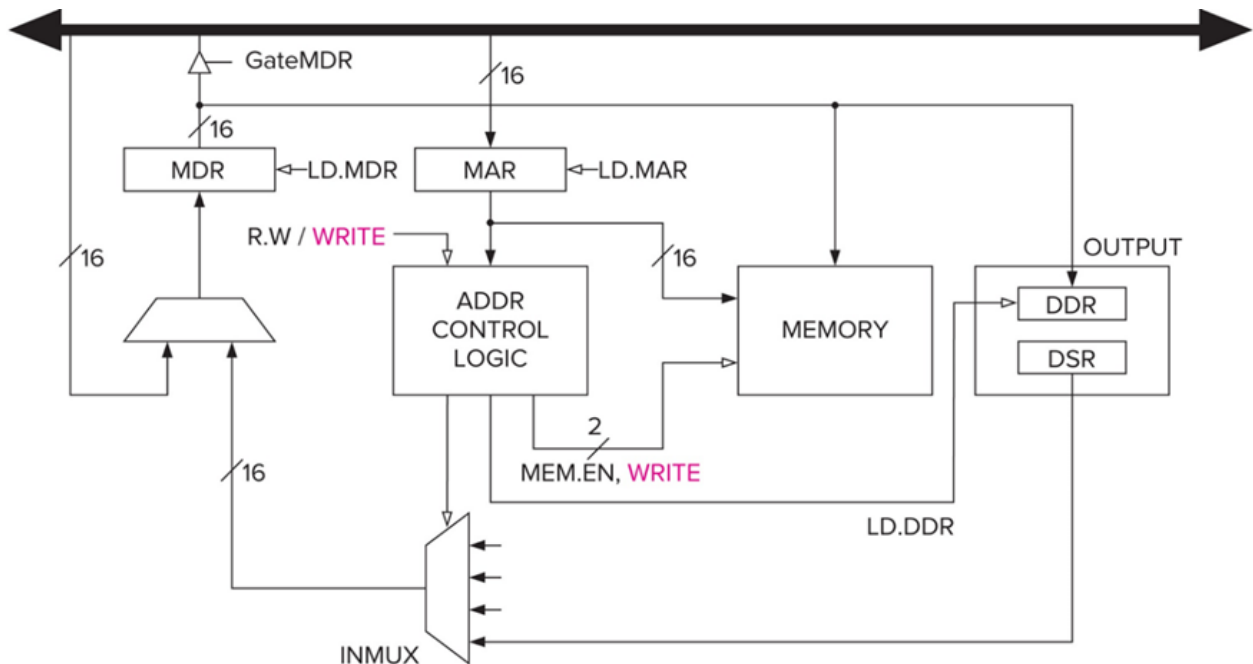
Normal data movement instructions are used with the memory-mapped I/O. The MAR and the MDR are used just like when reading from memory. The processor writes the address of the storage

location to be read into the MAR. This address, in the case of the keyboard input operation, is the address of the KBDR. The address control logic block decides where to read the data based on the address in the MAR. If the address is from the range allocated for addressing memory locations, the data is read from memory. In this case, though, since the address is that of the KBDR, so the address control logic decides to read the data from the Input device – the keyboard. The read data is then copied into the MDR, just like in the case of a memory read operation.



Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond)*, McGraw Hill, 3rd Edition

The output operation in LC-3 using the display monitor works similarly to the input operation. The address of the storage location where the data is to be written is put into the MAR, while the actual data is written into the MDR. Depending upon the address in the MAR, the Address Control Logic decides where the data is to be written, whether in a memory location or in the DDR of the output display device. In summary, for memory-mapped I/O to work, the LC-3 requires the use of the address control logic block to decode the address based on the allocations made in the memory address space.

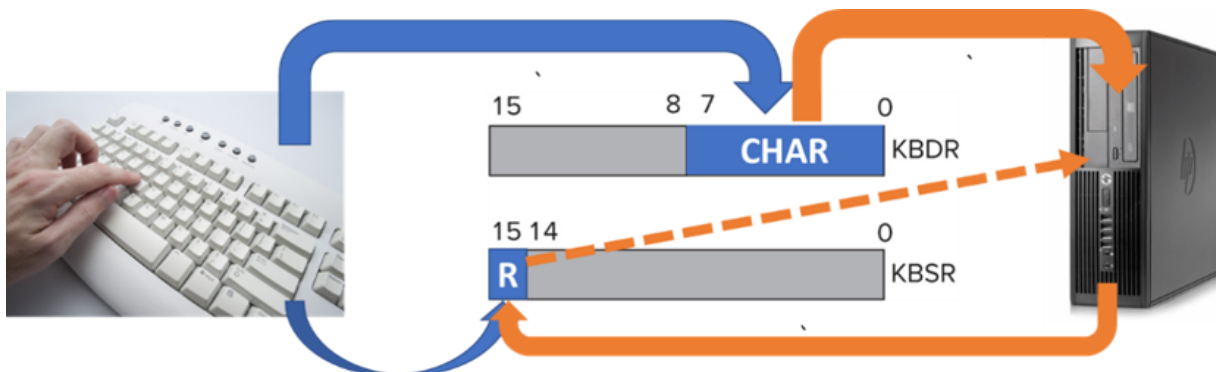


Source: Patt, Yale N., and Sanjay J. Patel, *Introduction to Computing Systems (From Bits and Gates to C/C++ and Beyond)*, McGraw Hill, 3rd Edition

Handling Asynchronous Data Transfer:

Synchronous data transfer happens when the data input occurs at fixed time intervals. In this case, the CPU knows exactly when it should read the input data register to get new data. Unfortunately, I/O operations are not always synchronous. When there is no predictability in when new input may come, this is called a synchronous data transfer. Due to the lack of predictability of when new input data may appear, the CPU cannot decide when to read the input data.

Fortunately, a solution exists, the ready bit or the R bit, which exists in the LC-3 KBSR and DSR registers. The ready bit can be used for synchronization between the CPU and the input/output device.



Consider a typist typing using a keyboard. When the typist presses a key, the keyboard device has a character to input into the keyboard data register. As the input character is written into the KBDR, the Ready bit of the KBSR is Set. This bit is an indication to the CPU that there is fresh input data in the KBDR. When the CPU checks and finds the Ready bit set, it can proceed to read the KBDR. As the CPU reads the new input character from the KBDR, it also goes ahead and resets the Ready bit in the KBSR. Now until the ready-bit is set again, the CPU knows that there is no new input data to read. And as soon as the Ready bit is set again, the CPU can once again read the new data. This is

how asynchronous data transfer works, the device and the CPU synchronize for every data transfer, using the Ready-bit.

The asynchronous output operation, where the CPU sends asynchronous data to the display monitor work similarly, with the ready-bit of the DSR used for the same purpose of synchronization between the CPU and the display device.

Data Transfer Schemes

Programmed I/O:

When the data transfer is handled via the CPU, the CPU has direct control over the I/O operation. The CPU has the ability to sense or read the status information and send the read or write commands. So, when the CPU encounters an I/O-related instruction inside a software program, it issues an I/O command. Alongside this, it specifies an address that identifies the I/O module and the external device to which the I/O command relates. The I/O module then communicates with the External I/O device to implement the I/O command from the CPU. Once the I/O operation is performed with the external device, the I/O module sets the appropriate bits in the status register, which the CPU can thereafter check or read.

There are four types of I/O commands that the CPU can send to the I/O module, and these are Control, Test, Read and Write. Control commands are specific to the type of external I/O device, and these commands tell the device to perform some control action – for example, the CPU may ask an external storage disk to spin itself up. Test commands are used to test the status conditions of the device—for example, to test if the device is powered on or if any errors occurred during the last I/O operation. Read command is a command from the CPU to the I/O module to read data from the external device and place it in the data register on the I/O module, from where the processor can read it via the data bus. Lastly, the Write command from the CPU instructs the I/O module to read the data from the data bus with the CPU and then send it to the external device.

The single-bit status information in the form of the Ready bit is sufficient for the CPU to know that there is new data available in the KBDR to read, but the question is: how does the CPU get to know that the R bit is set in the KBSR in the first place? Does the I/O module inform the CPU when the R-bit gets set? In programmed I/O, the I/O module has no responsibility to inform the CPU after the setting of the status information. Neither does it interrupt the normal processing of the CPU. The I/O module just sets the status information and waits for the CPU to discover it. In the programmed I/O technique, the CPU has the responsibility to check the status register periodically—we can say that the CPU polls the status bits periodically.

The disadvantage of using programmed I/O is the inefficiency it brings to the entire I/O process. Due to a lack of certainty on when the I/O operation may complete, programmed I/O causes the CPU to keep polling the status registers periodically, leading to unnecessary wastage of the CPU cycles, which could have been used for other meaningful purposes. It results in the CPU remaining unnecessarily busy and is, therefore, the biggest disadvantage of the technique.

Interrupt-Driven I/O:

Similar to the programmed I/O technique, the data transfer in the interrupt-driven I/O technique is also coordinated through the processor or the CPU. But unlike the programmed I/O technique, where the processor polls the status registers periodically, the Interrupt-driven I/O works on the principle of Interrupts.

When the CPU encounters an I/O-related instruction inside a software program, it issues an I/O command to the I/O module. Let us take the example of a READ command. Once the processor has issued the READ command to the I/O module, the processor goes back to doing other meaningful work, leaving the I/O module to coordinate and work on the READ operation. The I/O module interacts with the external device to fetch the read data and stores it in the data register. Once the I/O module has the data in its data register, it notifies the processor that the data is ready to be read. This is done in the form of an interrupt issue by the I/O module to the processor. Thus, there is no

need for the processor to continuously poll for the change in the state of the status registers. Rather, it is the responsibility of the I/O module to interrupt the processor when it is ready with the data that the processor wanted to read. When the CPU receives this interrupt, it enters the interrupt handling state.

The CPU handles the interrupt by requesting the data from the I/O module, which then transfers this data toward the CPU over the system bus. Once the CPU reads the data, it exits the interrupt handling state and goes back to its normal execution. From the perspective of the software program that was interrupted briefly by the I/O interrupt, there is not much of a perceivable difference in execution, except for a slight delay in the completion of execution.

The biggest advantage of the interrupt-driven I/O technique over the programmed I/O technique is primarily due to the fact that the CPU does not have to waste its previous cycles executing the polling procedure and checking for status registers. On the contrary, it can utilize the time for doing some other productive work, maybe even work on another problem or software program. Once the I/O operation is completed by the I/O module, the CPU is anyway notified via the interrupt. So, basically, it is not as if there is any additional delay encountered to the I/O operation, and yet, the CPU could perform some other meaningful work in the meantime.

The only disadvantage of the Interrupt-driven I/O technique is that, like the programmed I/O technique, it still requires the use of some CPU cycles, those that are used during data transfer from the external device to the memory. This disadvantage does not make the interrupt-driven I/O technique any worse than the programmed I/O technique, which behaves similarly.

The interrupt is a mechanism that is used by the computer system for handling various events, not just I/O events. These could be a power failure event, a timer expiry event, or others. There are two parts to an interrupt mechanism: a mechanism for the I/O device to interrupt the CPU and a mechanism that handles the Interrupt Request.

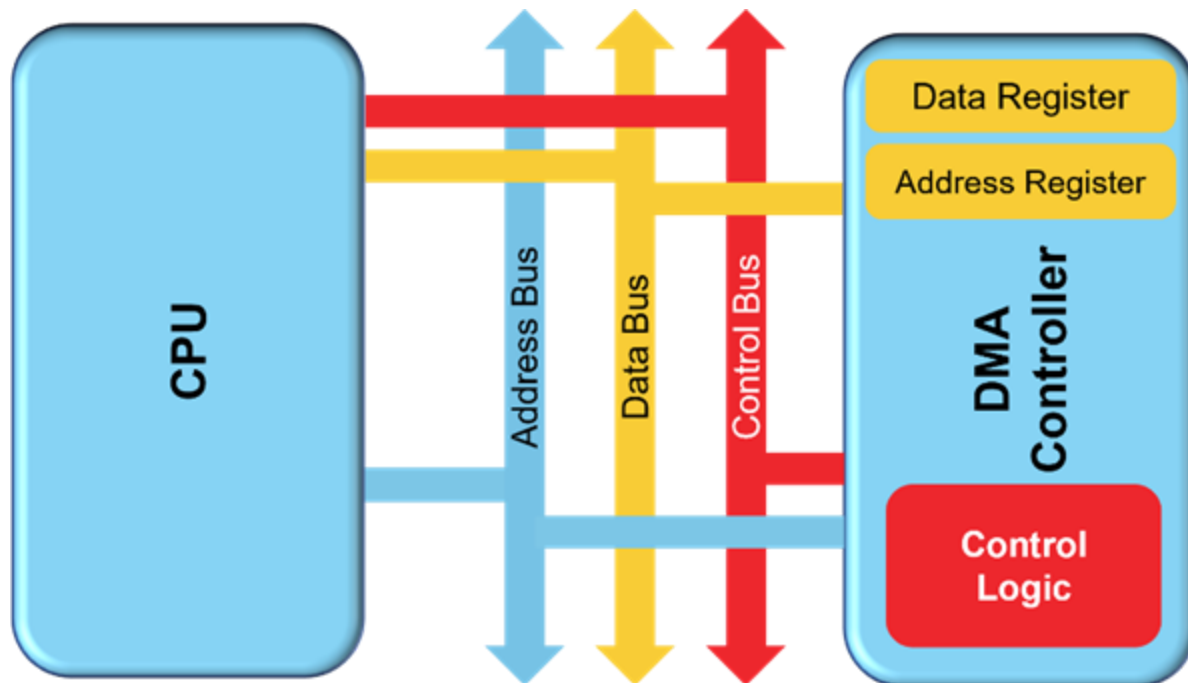
Direct Memory Access (or DMA):

Both Interrupt-driven and Programmed I/O Techniques require active intervention from the CPU for I/O data transfers. While the interrupt-driven I/O technique does utilize CPU cycles more efficiently than Programmed I/O, in both techniques, the actual data transfer from the I/O module to the memory is still performed by the CPU. This results in two key disadvantages:

For one, this limits the rate at which data transfer can happen. The overall I/O transfer rate gets limited by the speed at which the CPU can service the I/O operation. Secondly, since the CPU is actively involved during the data transfer process, it keeps the CPU busy. In fact, the CPU must process a number of instructions for executing the I/O data transfer.

We do not necessarily care about these limitations for a few words of data transfers. But for large data transfers, it really becomes quite inefficient. An improvement is possible only with some entity taking over the data transfer responsibility of the CPU and performing it on behalf of the CPU – something like a CPU surrogate. This is where direct memory access (or DMA) comes in.

In the DMA-based I/O technique, an additional hardware module called the DMA controller is introduced in the computer system, and it connects over the system bus to the CPU and the memory unit. The DMA controller acts as the CPU surrogate in performing the actual data transfer from the I/O module to the memory unit. It works under the control of the CPU—the CPU commands the DMA controller to perform the read/write I/O operations, providing the DMA controller with enough information to effectively handle a large data transfer.



To list it out, the CPU provides the following information to the DMA controller:

- It tells the DMA controller whether the I/O operation to be performed is a Read or a Write Operation.
- It provides the address of the I/O Device with which the I/O operation is to be performed.
- Next, the CPU informs of the starting address inside the memory unit where the data read from the I/O device is to be written OR where the data is to be read from to send to the I/O device.
- Lastly, the CPU tells the DMA controller how much data is to be read or written as part of the I/O operation.

With all this information from the CPU, the DMA controller can now function on its own, performing the same function as the CPU does to complete the I/O data transfer with the I/O module.

Meanwhile, while the DMA controller deals with the data transfer, the CPU can go back to doing other meaningful work. Once the DMA controller is done with the data transfer between the I/O module and the memory unit, it sends an interrupt to the CPU, informing it of the completion of the I/O operation.

Since the DMA controller, the CPU, and the memory unit are connected via the system bus. The DMA controller must share the system bus with the CPU and vice-versa. While the CPU is busy executing some other program, it uses the system bus to fetch instructions, fetch operands, or store the result, as part of the instruction cycle. At the same time, the DMA controller may need the system bus to transfer data to or from the memory unit for the purpose of the I/O operation that is ongoing simultaneously. How does the DMA controller share the system bus with the CPU?

There are two possible options for the DMA controller to get access to the system bus. The simple option is for the DMA controller to wait for the system bus to become free and use it when the CPU does not require the system bus. However, the alternate and more often used option is to force the CPU to suspend its operations temporarily. This approach is called **cycle stealing** since the DMA controller steals a bus cycle from the processor. If the CPU is suspended just before it needs to use the bus, the DMA module transfers one word using a bus cycle and returns the control back to the

processor. Thus, the DMA steals bus cycles to transfer a word of information. This causes some reduction in the speed of execution of the software program running on the CPU, but the degradation in performance is not as high as in Programmed I/O or even in Interrupt-driven I/O.